FLYRANK.AI · AI BACKEND ENGINEERING · PROJECT BRIEF

Connecting your CRUD to the database

Take your in-memory CRUD API and move its storage to a real SQLite database — same endpoints, but now your data survives a restart.

~4 h across 6 stages | Stretch +2 h | Bonus AI stage +1 h | JavaScript or Python | $0 · no credit card

RELATED GUIDE
The developer survival kit

YOU WILL PRACTICE
SQLite · SQL queries · persistence · parameterized queries · DB Browser

DELIVERABLE
Public GitHub repo, ≥6 commits, README + DB screenshot

How to read this document: new words are shown in **bold** the first time they appear — every one of them is explained in the Glossary at the end. Work the stages **in order**; each ends with a checkpoint that proves it works. This project is the direct sequel to **Build your first CRUD API**; you continue in the same repo and the same lane.

Contents

1. Goal & purpose 1
2. The big idea in 60 seconds 2
3. Tools — pick one lane 2
4. The task — six stages 2
5. Bonus stage — the AI rematch 6
6. Requirements & stretch goals 6
7. Done means 7
8. Curated resources 8
9. Glossary 11

1 · Goal & purpose

Goal: take the CRUD **API** you built earlier and replace the in-memory task list with a real **SQLite database**. Your endpoints behave exactly the same — but now your data survives when the server restarts, because it lives in a file called `tasks.db` instead of in your program's memory.

Until now, your tasks disappeared every time you restarted the server. That wasn't a bug — it was the limitation of storing data in memory, and a database is exactly what fixes it. Real applications keep their data in a **database**: instead of a list living inside your code, your server now saves each task to disk, so it's still there tomorrow. That single change — memory to disk — is what turns your project from a demo into something real.

Here is the exciting part: **almost none of your API changes.** Clients send the same requests to the same endpoints and get the same responses back. Only the **storage layer** underneath changes. The **API** describes *what* your application does; the **database** describes *where* it keeps its data. A shop's checkout works this way, and so does FlyRank: the endpoint that returns an SEO report looks identical whether the report sits in memory, in SQLite, or in a giant Postgres cluster.

Beginners usually expect this to be a big rewrite. It isn't — you change your storage code and leave your routes alone, and most of your "aha" moments come from watching the API behave **exactly as before**. Golden rule: if you find yourself rewriting your endpoints, stop — only the storage code should be changing.

2 · The big idea in 60 seconds

Before, your architecture was a client talking to an API talking to a list in memory. Now it becomes a client talking to the same API talking to a SQLite database:

```
Before:        Client  ->  API  ->  a list in memory
This one:      Client  ->  API  ->  SQLite database (tasks.db)
```

The client cannot tell the difference. Line them up:

Layer	In-memory	With SQLite	Does the client notice?
Client	sends `GET /tasks`, `POST /tasks`…	sends the **same** requests	—
API / routes	`GET /tasks` returns tasks	`GET /tasks` returns tasks	**No**
Storage	a list in a variable	rows in `tasks.db`	**No**
After a restart	data is gone	**data is still there**	Only *you* notice

The API is the promise; the database is where the promise is kept. **Swapping the storage keeps the promise** — that separation is the whole project. Everything below just moves one layer from memory to disk, one endpoint at a time.

3 · Tools — pick ONE lane

Both lanes build exactly the same thing. **Stay in the lane you chose before** — this is the same repo growing, not a new project.

	JavaScript lane	Python lane
Language	Node.js (free, nodejs.org)	Python 3.10+ (free, python.org)
Your app	Reuse your Express API	Reuse your FastAPI app
Database library	better-sqlite3 — one `npm install`, synchronous, simple to read	`sqlite3` — **built into Python**, nothing to install — or SQLModel for an ORM
Database file	`tasks.db` (created automatically)	`tasks.db` (created automatically)
Viewer	DB Browser for SQLite (free)	same
Testing your API	curl + Hoppscotch (both free)	same
Publishing	Git + a free GitHub account	same

> **SQLite needs no server and no install of its own** — it's just a file on your computer, created for you the first time your app runs. Not sure which library? Python's `sqlite3` is already in the standard library (import and go); JavaScript's `better-sqlite3` is the friendliest option because its queries are synchronous — no `await`, code that reads top to bottom.

4 · The task — six stages (+ one bonus)

Work stage by stage, **in order**. Each stage ends with a checkpoint you can run to prove it works. Commit to Git after every stage (that's your ≥6 commits, honestly earned). **If you only finish Stage 3, push anyway — a working half is worth more than a broken whole.**

0 Create your database

~30 min

Before you can store anything, you need a shelf to store it on.

1. Install your lane's tool: **JS** `npm install better-sqlite3` · **Py** nothing to install, `sqlite3` ships with Python (SQLModel users: `pip install sqlmodel`).
2. In your code, open (and therefore create) a database file named `tasks.db`. Opening a SQLite file that doesn't exist yet creates it — that's your database.
3. Create a **table** named `tasks` **if it does not already exist**, with three **columns**: `id` (integer, **primary key** — the database hands out ids for you), `title` (text), and `done` (boolean, stored as `0` / `1`).
4. **Seed** three example tasks — but *only if the table is empty*. Count the rows first; insert the examples only when the count is `0`. This is what stops your examples from multiplying every time you start up.

> **CHECKPOINT** — restart your application three times. `GET /tasks` (or opening `tasks.db` in DB Browser) shows **exactly three** tasks — not six, not nine. The table survives restarts and the seed runs only once.

Commit: `Stage 0: create SQLite database`

1 Read from the database

~45 min

Your data is on the shelf. Now teach the API to read it from there instead of from memory.

1. Replace the code behind `GET /tasks` so it runs a **SQL query** — `SELECT * FROM tasks` — and returns whatever the database gives back.
2. Replace `GET /tasks/{id}` so it fetches **one** row: `SELECT * FROM tasks WHERE id = ?`. The `?` is a **parameterized query** placeholder — you pass the id separately and never glue it into the string yourself (that habit is what keeps databases safe).
3. Unknown ids still return **404** with `{ "error": "Task not found" }`, exactly like before. Only where the data comes from changes.

> **CHECKPOINT** — `curl -i http://localhost:3000/tasks` → `200` + your three seeded tasks, read live from `tasks.db`. `curl -i http://localhost:3000/tasks/999` → `404` + the error JSON.

Commit: `Stage 1: database read endpoints`

2 Create new tasks

~45 min

A new task arrives — and this time it's written down for good.

1. Change `POST /tasks` so it runs an `INSERT INTO tasks (title, done) VALUES (?, ?)` instead of pushing onto a list. Let the database assign the `id`; set `done` to `false` / `0`.
2. Keep the **same validation** as before: a missing or empty `title` still returns **400**; a successful create still returns **201** with the new task (including the id the database gave it).

> **CHECKPOINT** — create a couple of tasks, then **stop your server and start it again**, then run `GET /tasks`. The tasks you created are **still there.** Stop and notice: this is the first time your data has ever survived a restart. That is the entire point of a database.

Commit: `Stage 2: insert into database`

3 Update and delete

~45 min

Tasks get finished, tasks get cancelled — and the database remembers both.

1. Change `PUT /tasks/{id}` to run `UPDATE tasks SET title = ?, done = ? WHERE id = ?`. Return the updated task. Unknown id → `404`; invalid body → `400`.
2. Change `DELETE /tasks/{id}` to run `DELETE FROM tasks WHERE id = ?`. Unknown id → `404`; on success return `204` with an empty body, just like before.
3. **Stop and notice:** you now have a complete, database-backed CRUD API. Your data outlives your program — every serious backend on Earth is this idea, wearing more clothes.

> **CHECKPOINT** — create a task, mark it done with `PUT`, then `DELETE` it, confirming each step with `GET /tasks`. Restart the server between two steps and watch the state hold. All the right status codes appear (`201`, `200`, `204`, `404`).

Commit: `Stage 3: update and delete with SQL`

4 Learn your first SQL by hand

~45 min

You've been sending SQL through your code. Now open the database and talk to it directly.

1. Open `tasks.db` in DB Browser for SQLite (free). You'll see your `tasks` table and its rows — the same data your API serves, laid out like a spreadsheet.
2. In its "Execute SQL" tab, run each of these by hand and read what comes back:

```
SELECT * FROM tasks;                -- list every task
SELECT * FROM tasks WHERE done = 1;  -- only completed tasks
SELECT COUNT(*) FROM tasks;          -- how many tasks are there?
UPDATE tasks SET done = 1;           -- mark every task completed
DELETE FROM tasks WHERE done = 1;    -- delete all completed tasks
```

3. After a query that changes data, call `GET /tasks` from your API. **Your API reflects the change instantly** — because the API and DB Browser read the exact same file. There is no "syncing"; there is one source of truth.

> **CHECKPOINT** — you changed the database by hand in DB Browser and then saw that change appear through your API, with no server restart. Save one query you ran and one sentence on what it returned for your README.

Commit: `Stage 4: explored SQLite`

5 Publish your database project

~30 min

Your work only counts when someone else can clone it and it just runs.

1. Push your updated project to your **public GitHub repo** (the same repo as before — its ≥6 stage commits come with it).
2. Update your **README** to add: why **SQLite** was chosen (single file, zero setup, survives restarts); where the database file lives (`tasks.db` , created automatically, usually git-ignored so each clone starts fresh); one documented command to start the project; a **screenshot** of your database open in DB Browser; and one example SQL query you ran in Stage 4.
3. Make sure the database is **created automatically** — a stranger who clones your repo and runs your one command gets a working app with its table and three seeded tasks, no manual setup.

> **CHECKPOINT** — on a clean clone (or delete `tasks.db` and restart), your one documented command starts the server, `tasks.db` is created automatically, and `GET /tasks` returns the three example tasks. A stranger could do this in under 5 minutes.

Commit: `Stage 5: database documentation` — then push everything.

★ Make it yours — optional extras

optional · as many or as few as you like

Now that queries are cheap, let the database do the work your code used to do.

None of these are required. Pick whatever sounds fun (creative alternatives welcome):

- **Search with SQL:** `GET /tasks?search=CRUD` using SQL's **LIKE** operator (`WHERE title LIKE ?`) — filtering in the database, not in a loop.
- **Filter by status:** `GET /tasks?done=true` using a `WHERE done = ?` clause.
- **Sort alphabetically:** return tasks ordered by title with `ORDER BY title` .
- **Real statistics:** `GET /stats` computed with `SELECT COUNT(*)` in SQL instead of counting in your code.
- **Timestamps:** add `created_at` and `updated_at` columns and set them on insert/update. In your README, write two sentences on what changing the table's shape felt like — *that feeling is why **migrations** exist, which you'll meet properly later on.*

Commit (if you build any): `Extras: <what you added>`

5 · Bonus stage — the AI rematch

6 The AI rematch

~1 h · optional — and the most fun

You just moved a whole API onto a database by hand. Now hire the fastest junior developer on Earth to do the same migration — and review their work.

You did Stages 0–5 by hand for a reason: you now know exactly what a correct memory-to-SQLite swap looks like. That knowledge is what turns this stage from a magic show into a **code review**.

1. **Write the prompt yourself — this is the real exercise.** Without copying from this document, write your own **prompt** asking an AI assistant to move an in-memory CRUD task API to SQLite. From memory, specify what matters: your lane and library, the `tasks` table's columns, "create the table if missing," "seed three tasks only when empty," the five endpoints keeping identical behaviour, the `400` / `404` rules, and **parameterized queries** for safety.
2. **Generate in quarantine.** Put the AI's version in a separate folder (`ai-version/`) or branch. Your Stages 0–5 code stays untouched — that hand-built version is your deliverable.
3. **Run it.** Does it start on the first try and create its `tasks.db` ? Fire your Stage 2 and Stage 3 checkpoints at it. Does its seed run only once, or do the examples multiply on restart? Does data actually survive a restart?
4. **Diff it.** Compare the AI's storage code with yours (`git diff --no-index your-file ai-file`). Then answer three questions in an **"AI vs me"** section of your README: What did it do *better* — and can you explain it (a transaction, a smarter schema, cleaner parameter binding)? What did it get *wrong* or quietly ignore (seeds that multiply, string-glued SQL, a changed status code, an invented column type)? What did *your prompt* forget to specify — and what did the AI silently decide for you?
5. **One rematch.** Improve your prompt with what you learned, regenerate, and note in one sentence what changed.

> The lesson hiding in this stage: an AI's output is exactly as good as your specification — and you could only judge it because you built the thing yourself first. **Both halves of that sentence are your career from now on.**

> **CHECKPOINT** — your README has an "AI vs me" section containing your full prompt and at least three concrete differences you found.

Commit: `Stage 6: AI vs me` (AI code stays in its own folder/branch).

6 · Requirements

Done = every box ticked. Each one is checkable in under a minute.

- [ ] The API exposes the **same CRUD endpoints** as before — `GET /tasks` , `GET /tasks/:id` , `POST /tasks` , `PUT /tasks/:id` , `DELETE /tasks/:id` — with the same request/response shapes.
- [ ] Tasks are stored in **SQLite** (`tasks.db`), not in memory.
- [ ] **Data survives a server restart** (shown two ways: via the API, and by opening `tasks.db` in DB Browser).
- [ ] The **database file** is created automatically if it's missing.
- [ ] The **tasks table** is created automatically if it's missing.
- [ ] **Three example tasks are seeded only on the first run** — restarting does not duplicate them.
- [ ] All CRUD operations use **SQL queries** with **parameterized** placeholders (no user input glued into SQL strings).
- [ ] Correct status codes carried over from before: `200` / `201` / `204` success, `400` invalid body, `404` unknown id — each error with a JSON `error` message.
- [ ] Public **GitHub repo** updated: README with why-SQLite, run command, one example SQL query, and a **DB Browser screenshot**.

Stretch (optional)

- [ ] **Prove the API didn't change:** keep your endpoint tests (or the same curl commands) and show they still pass against the SQLite version — then explain in your README why identical tests passing is the proof that storage is "just an implementation detail."
- [ ] **Add an index** on the column your search/filter extra queries, and write one line on what an index is for.
- [ ] **Wrap a multi-step change in a transaction** (e.g. seeding the three tasks) so it's all-or-nothing, and explain in one line why that matters.
- [ ] Stage 6 — the AI rematch (see above): prompt an AI to do the same migration, run it, diff it, write your "AI vs me" section.

7 · Done means

- The full CRUD cycle works against the database, shown **two ways**: once via `curl -i` (right status codes visible), and once by opening `tasks.db` in DB Browser and seeing the same rows.
- **Persistence is proven:** create tasks, restart the server, and they're still there.
- Your repo is public, the README explains the SQLite choice and runs on a clean clone (the database creates itself), and `git log` shows one honest commit per stage.

8 · Curated resources

Don't read everything. Each section says *when* you need it. All resources are free, no credit card. Rows marked JS are JavaScript-lane, Py are Python-lane.

● **Start here** (everyone) · ● **When you're comfortable** · ● **Optional deep dive**

§1 · What SQLite is — before you start

	Resource	Format	Why it's useful
●	What is SQLite? (SQLite Tutorial)	Article, ~8 min	The plainest explanation: a serverless, zero-config database that lives in one file. Read before Stage 0.
●	When to use SQLite (official)	Official, ~10 min	Straight from the source: what SQLite is great at and when you'd reach for Postgres. Answers the "why SQLite?" your README asks for.
●	Codecademy — What is SQLite	Article, ~10 min	A second explanation in different words if the first didn't click — with where SQLite shows up in the real world.

§2 · Your database library — official docs

	Resource	Format	Why it's useful
●	JS better-sqlite3 (npm)	Package docs	The recommended JS library. Shows `new Database('tasks.db')`, `.prepare()`, `.run()`, `.get()`, `.all()` — the calls you need for all of CRUD. Synchronous.
●	Py Python sqlite3 — docs + tutorial	Official docs	Built into Python. The tutorial walks connect → create table → insert → select → persist — Stages 0–2 almost exactly.
●	Py SQLModel	Official docs	The ORM option (by FastAPI's author). Use it if you'd rather define a `Task` model than write SQL strings.

§3 · Build-along tutorials — watch the migration get coded

Watching or typing along is allowed and encouraged — just make the final code yours (your fields, your validation, your extras).

	Resource	Format	Why it's useful
●	JS REST API with Node, Express & SQLite	Article, ~17 min	Builds an Express API backed by better-sqlite3 end to end — the closest worked example to this project for the JS lane.
●	Py FastAPI — SQL (Relational) Databases	Official tutorial	The official FastAPI tutorial that swaps in a SQLite database with SQLModel and does full CRUD on the same endpoints. Your Python-lane spine.
●	Py freeCodeCamp — Work with SQLite in Python	Handbook	A calm, thorough walk through raw `sqlite3`: connections, cursors, parameterized queries, committing. Read it if you chose plain `sqlite3`.
●	Py SQLite Databases with Python — Full Course	Video, ~1 h	freeCodeCamp's free video walkthrough of SQLite from Python — tables, inserts, queries, parameters. Watch a stage ahead, then build it.

§4 · SQL by hand — for Stage 4 and the extras

	Resource	Format	Why it's useful
●	DB Browser for SQLite	Free app	The free visual tool Stage 4 uses: open `tasks.db`, browse rows, run SQL by hand. Win/macOS/Linux. Grab it before Stage 4.
●	SQLBolt — interactive SQL lessons	Interactive, ~1 h	Learn SELECT, WHERE, ORDER BY, INSERT, UPDATE, DELETE in the browser with instant feedback. The fastest way to be comfortable with the Stage 4 queries.
●	Internal — SQL basics guide	Internal guide	Our one-page survival guide: the query shapes you'll write 100 times, plus the traps (missing WHERE, string-glued SQL) this program tests you on.
●	SQLite Tutorial — SELECT, LIKE, ORDER BY	Reference site	SQLite-specific reference for when an extra needs a clause you haven't met — LIKE for search, ORDER BY for sorting, COUNT() for stats.

§5 · Keep it safe — parameterized queries

	Resource	Format	Why it's useful
●	Bobby Tables — preventing SQL injection	Reference, ~10 min	Shows *why* you pass values as `?` parameters instead of gluing them into SQL, with the fix for every major language. Read the short intro; keep your lane's snippet handy.

§6 · Git & GitHub — publish your work (Stage 5)

	Resource	Format	Why it's useful
●	Internal — Git & GitHub guide	Internal guide	The 20% you use daily: add → commit → push, plus `.gitignore` (git-ignore `tasks.db` so each clone starts fresh). Everything Stage 5 needs.
●	GitHub Docs — Hello World	Official, ~15 min	If your repo setup is hazy, the fastest refresher on repos, commits, and pushing to GitHub.
●	How to write a good README	Article, ~15 min	Stage 5 asks for a real README (why-SQLite, run command, screenshot). This shows what a good one contains.

Suggested path for a complete beginner

1. Before Stage 0: section §1 (both green reads) — ~15 min, so you know what SQLite is.
2. Stages 0–3: keep your lane's build-along (§3) and library docs (§2) open; build one stage, then the next.
3. Stage 4: install DB Browser (§4), and if SQL feels shaky do a few SQLBolt lessons first.
4. Stage 5: the internal Git guide (§6) — commit, then push.
5. Curious or stuck: the yellow rows. Everything red is optional forever.

Per-stage critical resources

Stage	Keep these open
0–3 · build the DB	Your lane's build-along (§3) + library docs (§2)
4 · SQL by hand	DB Browser + SQLBolt + internal SQL basics guide (§4)
5 · publish	Internal Git & GitHub guide (§6)
Any time · "why SQLite?"	§1 (what SQLite is and when to use it)

9 · Glossary

Plain-language definitions of every **bold** word above. No definition depends on another — read them in any order.

Word	What it means
API	The set of doors your program offers so other programs can talk to it — here, the five task endpoints. It stayed the same this whole project; only what's behind it changed.
Database	A system that stores data on disk so it's still there after a program stops and starts again. Your to-do list now lives in one.
SQLite	A lightweight database that is just a single file on your computer, with no separate server to install or run. Your whole database is the file `tasks.db`.
Storage layer	The part of your code responsible for keeping and fetching data. With SQLite swaps the storage layer (memory to SQLite) while leaving the rest of the app alone.
Table	A collection of related data arranged in rows and columns, like one sheet in a spreadsheet. You made one table called `tasks`.
Row	One record in a table. In this project, one task is one row.
Column	One named, typed property stored for every row — here `id`, `title`, and `done`.
Primary key	A column whose value uniquely identifies each row. `id` is the primary key of `tasks`; SQLite fills it in for you.
SQL	Structured Query Language — the language for creating, reading, updating, and deleting data in a database. `SELECT * FROM tasks` is SQL.
Query	One SQL command sent to a database. `SELECT`, `INSERT`, `UPDATE`, and `DELETE` are the four you use here.
Parameterized query	A query where you leave a placeholder (`?`) for user input and pass the value separately, instead of pasting it into the SQL text. It's how you keep user data from breaking or attacking your database.
Seed	To insert some starting example data the first time an app runs — here, the three example tasks, inserted only when the table is empty.

Persistence	The quality of data staying available after the program stops and starts. Memory has none; a database gives you persistence.
Schema	The shape of a database: which tables exist and which columns they have. Your schema is one `tasks` table with `id`, `title`, `done`.
Migration	A controlled, written-down change to a database's schema as a project grows (for example, adding a `created_at` column). You'll work with these properly later on.
CRUD	Create, Read, Update, Delete — the four basic things an app does with its data, and the four endpoints you moved onto the database.
Validation / validate	Checking incoming data before trusting it (is `title` present and non-empty?) and rejecting bad input with a `400`. Carried over unchanged from before.
Status code	The 3-digit number in every response saying how it went: `200` OK, `201` Created, `204` No Content, `400` Bad Request, `404` Not Found.

LIKE	A SQL operator for matching text patterns — used in the search extra: `WHERE title LIKE '%CRUD%'` finds every task whose title contains "CRUD".
Prompt	The instructions you give an AI assistant. In Stage 6 your prompt is a mini-specification of the whole migration — the more precisely it names the table, rules, and endpoints, the closer the AI's output lands to yours.
Diff	A line-by-line comparison of two versions of code showing what was added, removed, or changed. `git diff` produces one; reading diffs is how professionals review each other's work.
Git / GitHub / repo / commit	Git tracks versions of your code in a **repo**sitory; a **commit** is one saved step with a message; GitHub hosts the repo online so others can clone and run it.
README	The front page of a repo: what the project is, how to run it, and how it works.

FlyRank.ai · AI Backend Engineering · Project Brief — Connecting your CRUD to the database. All linked resources are free with no credit card required.